

Fiche de révision Docker

Notions clés · Définitions · Commandes essentielles — CDA

1. Concepts fondamentaux

Docker — Plateforme de *conteneurisation* : emballer une application et toutes ses dépendances dans une unité isolée et portable.

Conteneur (container) — Instance *exécutable* et isolée d'une image. Léger, démarre en quelques secondes, partage le noyau de l'hôte.

Image — Modèle *en lecture seule* (read-only) servant à créer des conteneurs. Composée de couches (layers) empilées.

Dockerfile — Fichier texte de *recette* décrivant comment construire une image, instruction par instruction.

Layer (couche) — Chaque instruction d'un Dockerfile crée une couche. Mises en cache pour accélérer les builds.

Registry / Docker Hub — Dépôt distant stockant les images (ex. `hub.docker.com`). On y *push* et *pull* les images.

Volume — Stockage *persistant* géré par Docker, indépendant du cycle de vie du conteneur (données qui survivent).

Bind mount — Montage d'un dossier de l'hôte dans le conteneur (ex. votre code source en dev).

Network — Réseau virtuel permettant aux conteneurs de communiquer entre eux par leur nom.

Docker Compose — Outil pour définir et lancer une appli *multi-conteneurs* via un fichier YAML.

Conteneur vs VM : une VM virtualise tout un OS (lourd, lent). Un conteneur partage le noyau de l'hôte et n'isole que l'application (léger, rapide, quelques Mo).

2. Cycle de vie & images

Commande	Description
<code>docker pull <image></code>	Télécharger une image depuis le registry
<code>docker images</code>	Lister les images locales
<code>docker build -t mon-app .</code>	Construire une image depuis le Dockerfile du dossier courant
<code>docker rmi <image></code>	Supprimer une image
<code>docker tag src cible:tag</code>	Renommer / taguer une image
<code>docker push user/image:tag</code>	Envoyer une image vers un registry
<code>docker history <image></code>	Voir les couches d'une image

3. Gestion des conteneurs

Commande	Description
<code>docker run <image></code>	Créer + démarrer un conteneur depuis une image
<code>docker run -d</code>	<i>Detached</i> : en arrière-plan
<code>docker run -p 8080:80</code>	Publier un port (hôte:conteneur)
<code>docker run -it bash</code>	Mode interactif + terminal (ouvrir un shell)
<code>docker run --name web</code>	Nommer le conteneur
<code>docker ps</code>	Lister les conteneurs actifs
<code>docker ps -a</code>	Lister tous les conteneurs (même arrêtés)
<code>docker stop / start / restart <c></code>	Arrêter / démarrer / redémarrer
<code>docker rm <conteneur></code>	Supprimer un conteneur arrêté
<code>docker logs -f <conteneur></code>	Afficher les logs (<code>-f</code> = en continu)
<code>docker exec -it <c> bash</code>	Ouvrir un shell dans un conteneur déjà lancé
<code>docker inspect <c></code>	Détails complets (JSON) du conteneur
<code>docker stats</code>	Consommation CPU / RAM en temps réel

run vs exec : `run` crée un *nouveau* conteneur à partir d'une image. `exec` exécute une commande dans un conteneur qui *tourne déjà*.

4. Docker Compose

Commande	Description
<code>docker compose up</code>	Construire + démarrer tous les services
<code>docker compose up -d</code>	Démarrer en arrière-plan
<code>docker compose up --build</code>	Forcer la reconstruction des images
<code>docker compose down</code>	Arrêter + supprimer conteneurs et réseaux
<code>docker compose down -v</code>	... + supprimer aussi les volumes
<code>docker compose ps</code>	État des services
<code>docker compose logs -f <service></code>	Logs d'un service
<code>docker compose exec <svc> bash</code>	Shell dans un service
<code>docker compose build</code>	(Re)construire les images sans démarrer
<code>docker compose restart <svc></code>	Redémarrer un service

Structure type d'un docker-compose.yaml

```
services:
  app:
    build: ./docker/php      # build depuis un Dockerfile
    image: php:8.4-fpm-alpine # OU image existante
    ports:
      - '8080:80'           # hôte:conteneur
    volumes:
      - './src:/www'         # bind mount (code)
    environment:
      - APP_ENV=dev
    depends_on:
      - db                   # ordre de démarrage
volumes:
  db_data:                  # volume nommé (persistant)
```

5. Dockerfile — instructions clés

Instruction	Rôle
<code>FROM</code>	Image de base (obligatoire, 1re ligne)
<code>WORKDIR</code>	Définit le dossier de travail
<code>COPY</code> / <code>ADD</code>	Copier des fichiers de l'hôte vers l'image
<code>RUN</code>	Exécuter une commande au moment du build (ex. installer des paquets)
<code>CMD</code>	Commande par défaut au démarrage du conteneur (remplaçable)
<code>ENTRYPOINT</code>	Commande principale, non remplaçable facilement
<code>EXPOSE</code>	Documente le port utilisé (informatif)
<code>ENV</code>	Variable d'environnement
<code>ARG</code>	Variable disponible uniquement au build

RUN vs CMD : `RUN` s'exécute pendant la *construction* de l'image. `CMD` / `ENTRYPOINT` s'exécute au *lancement* du conteneur.

6. Volumes, réseaux & nettoyage

Volumes & réseaux

<code>docker volume ls</code>	Lister les volumes
<code>docker volume rm <v></code>	Supprimer un volume
<code>docker network ls</code>	Lister les réseaux
<code>docker -v hôte:conteneur</code>	Monter un volume

Nettoyage (libérer de l'espace)

<code>docker system prune</code>	Supprimer tout l'inutilisé
<code>docker system prune -a</code>	... + images non utilisées
<code>docker container prune</code>	Conteneurs arrêtés
<code>docker image prune</code>	Images orphelines

Astuce dépannage : si Docker monte un *dossier* à la place d'un *fichier* (erreur « not a directory »), c'est souvent qu'un chemin de bind mount n'existe pas → Docker le crée en tant que dossier. Vérifiez bien le chemin dans `volumes:` .